

Estrutura de Programas: Funções e Structs

Curso de Microcontroladores

O desafio de um programa grande

- Quando seu programa tem centenas de linhas, fica muito difícil de ler, manter e corrigir.
- Programas de robótica, como um seguidor de linha, precisam:
 - Ler sensores constantemente.
 - Fazer cálculos matemáticos complexos (como PID).
 - Controlar motores em tempo real.
 - Talvez comunicar via serial.
- Fazer tudo isso em um único bloco de código seria complexo e geraria um código difícil de manter.
- A solução é **quebrar o programa em partes menores e testáveis**.

O Padrão IPO (Entrada, Processamento, Saída)

- Todo programa segue logicamente um padrão simples: **IPO**.
- **Entrada**: O robô coleta dados do mundo físico (sensores).
- **Processamento**: O código analisa esses dados e toma decisões.
- **Saída**: O robô atua no mundo (motores, LEDs, etc).
- No caso do seguidor de linha:
 - **Entrada**: Lê a linha com sensores ópticos.
 - **Processamento**: Calcula o erro de posição e aplica PID.
 - **Saída**: Ajusta a velocidade dos motores.

O Padrão IPO (Entrada, Processamento, Saída)

- O padrão IPO não representa só o que o seguidor é capaz de fazer por inteiro, com os sensores sendo entrada, o PID sendo processamento e os motores sendo saída.
- Ele também pode ser aplicado dentro de cada coisa, ou seja, o sensor pode ser seu IPO, sendo a entrada a luz refletida, o processamento a conversão analógica-digital e a saída o valor 0-1023.
- O PID também tem seu próprio IPO, onde a entrada é o erro, o processamento é o cálculo dos termos P, I e D, e a saída é a correção.
- O controle dos motores também tem seu IPO, onde a entrada é a correção, o processamento é a validação e ajuste da velocidade, e a saída é o sinal PWM aplicado aos motores.
- Ou seja, o padrão IPO é uma forma de organizar o pensamento e o código em níveis hierárquicos, desde o programa inteiro até cada função individual.
- Isso torna tudo muito mais simples

loop()

```
void setup() {  
    // Executado APENAS uma vez ao ligar  
    Serial.begin(9600);  
    pinMode(SENSOR_PIN, INPUT);  
}  
  
void loop() {  
    // Executado REPETIDAMENTE, centenas de vezes por segundo  
    int leitura = analogRead(SENSOR_PIN);  
    Serial.println(leitura);  
    delay(10); // pequena pausa  
}
```

- 'setup()' roda uma única vez: você configura pinos, inicia comunicação serial, etc.
- 'loop()' roda infinitamente
- Quanto mais rápido o 'loop()' rodar, mais responsivo o robô será.

Por que usar funções?

- Sim, você *poderia* escrever tudo no 'loop()', mas seria pouco eficiente.
- Funções permitem que você **quebre o problema em partes menores**.
- Cada função possui uma responsabilidade única.
- Você testa cada função isoladamente antes de integrar.
- Exemplo: uma função apenas para ler sensores, outra apenas para calcular, outra para atuar.
- Isso também facilita trabalhar em equipe: um desenvolve 'lerSensores()', outro desenvolve 'calcularPID()'.

Exemplo: Um programa sem funções

```
void loop() {  
    // Ler 8 sensores manualmente  
    int s0 = analogRead(A0);  
    int s1 = analogRead(A1);  
    // ... mais 6 linhas ...  
    int s7 = analogRead(A7);  
  
    // Calcular a média e erro (muitas linhas)  
    int somatorio = s0 + s1 + ... + s7;  
    int media = somatorio / 8;  
    int erro = media - 500;  
  
    // Controlar motores manualmente  
    analogWrite(MOTOR_L, 200 - erro);  
    analogWrite(MOTOR_R, 200 + erro);  
}
```

- Tudo misturado no 'loop()'.
- Se houver um bug, é difícil saber se é na leitura, no cálculo ou na saída.
- Código muito repetitivo (muito 'analogRead()' seguido).

O que é uma função?

```
int somar(int a, int b) {  
    return a + b;  
}
```

```
void setup() {  
    int resultado = somar(5, 3); // resultado = 8  
}
```

- Função é um bloco de código reutilizável com um nome específico.
- 'int somar(int a, int b)' significa: recebe dois inteiros e **retorna** um inteiro.
- 'void' significa que a função não retorna nada (apenas executa).
- Parâmetros ('a' e 'b') são dados que você passa para a função.
- 'return' envia um valor de volta ao código que chamou a função.

Mini IPO: Função de Entrada

```
int lerSensores() {  
    // ENTRADA: coleta dados  
    int valores[8];  
    for (int i = 0; i < 8; i++) {  
        valores[i] = analogRead(sensorPinos[i]);  
    }  
  
    // PROCESSAMENTO: calcula a posição  
    int somatorio = 0;  
    for (int i = 0; i < 8; i++) {  
        somatorio += valores[i];  
    }  
    int posicao = somatorio / 8;  
  
    // SAÍDA (da função): retorna o resultado  
    return posicao;  
}
```

- Cada função pode ter seu próprio **mini IPO**.
- Esta função: lê sensores, processa, retorna um valor.
- Você pode testar 'lerSensores()' isoladamente para garantir que funciona.

Mini IPO: Função de Processamento

```
int calcularPID(int posicaoAtual) {  
    // ENTRADA: recebe a posição  
    static int erroAnterior = 0;  
    int erro = posicaoAtual - 500;  
  
    // PROCESSAMENTO: cálculo PID  
    float P = erro * 0.5;  
    float I = /* acumulador */ ;  
    float D = (erro - erroAnterior) * 0.2;  
    int correcao = (P + I + D);  
  
    erroAnterior = erro;  
  
    // SAÍDA: retorna a correção  
    return correcao;  
}
```

- Esta função recebe um valor, processa com PID, retorna a correção.
- Você pode testar com valores conhecidos: se posição é 500 (centro), deve retornar 0.

Mini IPO: Função de Saída

```
void controlarMotores(int velocidadeEsq,
                     int velocidadeDir) {
    // ENTRADA: recebe as velocidades (par metros)

    // PROCESSAMENTO: valida o
    velocidadeEsq = constrain(velocidadeEsq, 0, 255);
    velocidadeDir = constrain(velocidadeDir, 0, 255);

    // SAÍDA: atua nos motores
    analogWrite(MOTOR_L, velocidadeEsq);
    analogWrite(MOTOR_R, velocidadeDir);
}
```

- 'void' significa que não retorna nada, apenas executa.
- Recebe velocidades, valida, aplica aos pinos PWM.
- 'constrain()' garante que os valores fiquem entre 0-255.

O loop() como Orquestrador

```
void loop() {  
    // Mini IPO 1: Ler sensores  
    int posicao = lerSensores();  
  
    // Mini IPO 2: Calcular correção  
    int correcao = calcularPID(posicao);  
  
    // Mini IPO 3: Atuar nos motores  
    int velEsq = 150 - correcao;  
    int velDir = 150 + correcao;  
    controlarMotores(velEsq, velDir);  
}
```

- O 'loop()' não contém lógica complexa, apenas chamadas de função.
- Fica muito legível: você vê exatamente o que o robô faz em cada ciclo.
- Se há um problema, você sabe exatamente qual função investigar.
- Cada função roda rapidamente, mantendo a taxa de amostragem alta.

Estruturas: Agrupando Dados Relacionados

- Um robô tem muitos componentes: sensores, motores, LED, bateria, etc.
- Em vez de criar variáveis soltas ('motorEsqVel', 'motorDirVel', 'motorEsqPino'...), usamos 'struct'.
- 'struct' é um tipo de dado personalizado que agrupa valores relacionados.
- Exemplo: um motor possui pino PWM, pino de direção, velocidade atual. Todos esses dados relacionados podem estar em uma 'struct'.

Definindo e Usando uma Estrutura

```
struct Motor {  
    int pinoPWM;  
    int pinoDir;  
    int velocidade;  
};  
  
void setup() {  
    Motor motorEsq;  
    motorEsq.pinoPWM = 5;  
    motorEsq.pinoDir = 6;  
    motorEsq.velocidade = 0;  
  
    pinMode(motorEsq.pinoPWM, OUTPUT);  
    pinMode(motorEsq.pinoDir, OUTPUT);  
}
```

- Você acessa propriedades com o '.' (ponto).
- Deixa claro que essas três variáveis pertencem ao mesmo motor.
- Muito mais legível do que '*motor1_PWM*', '*motor1_DIR*', etc.

Funções que recebem estruturas

```
void controlarMotor(Motor motor, int velocidade) {  
    motor.velocidade = velocidade;  
    analogWrite(motor.pinoPWM, velocidade);  
  
    if (velocidade > 0) {  
        digitalWrite(motor.pinoDir, HIGH); // frente  
    } else {  
        digitalWrite(motor.pinoDir, LOW); // tr s  
    }  
}  
  
void loop() {  
    controlarMotor(motorEsq, 150);  
    controlarMotor(motorDir, 150);  
}
```

- Você passa uma 'struct' inteira para a função.
- A função trabalha com todos os dados do motor de uma vez.
- Muito mais elegante do que passar cada propriedade separadamente.

Exemplo Completo: Seguidor Estruturado

```
struct Sensor {
    int pino;
    int valor;
};

struct Motor {
    int pinoPWM;
    int pinoDir;
};

Sensor sensores[8];
Motor motorEsq, motorDir;

void setup() {
    // Inicializa sensores e motores
    for (int i = 0; i < 8; i++) {
        sensores[i].pino = A0 + i;
    }
    motorEsq.pinoPWM = 5;
    motorEsq.pinoDir = 6;
    // ... etc
```

Testando Cada Função Isoladamente

- Quando você quebra o programa em funções, pode testar cada uma separadamente.
- **Teste de entrada:** rode apenas `lerSensores()`, coloque `Serial.println()` e veja se os valores fazem sentido.
- **Teste de processamento:** passe valores conhecidos para `calcularPID()` e verifique se o resultado está correto.
- **Teste de saída:** rode `controlarMotores()` e ouça o motor para garantir que a velocidade está mudando.
- Isso torna muito mais fácil encontrar e corrigir bugs.
- Você descobrir que um motor não está funcionando é bem diferente de descobrir que o cálculo do PID está errado.

Laços (Loops) dentro de Funções

- Usar laços 'for' e 'while' dentro de funções é importante para controlar fluxo e repetição.
- Laços no 'loop()' do Arduino vs laços dentro de funções:
 - Laços dentro de funções têm **escopo local**, o que significa que as variáveis do laço não afetam o resto do programa.
 - Você tem mais controle: pode quebrar, continuar, ou fazer múltiplas iterações de forma clara.
- Exemplo: usar 'for' para ler todos os 8 sensores dentro de 'lerSensores()' é mais claro do que ter 8 linhas soltas no 'loop()'.

Cuidado com delay()

```
//      RUIM: delay() no loop() bloqueia tudo
void loop() {
    lerSensores();
    delay(100); // 100ms sem fazer nada!
    calcularPID();
}
```

```
//      BOM: usar millis() para n o bloquear
unsigned long tempoAnterior = 0;
void loop() {
    unsigned long tempoAtual = millis();
    if (tempoAtual - tempoAnterior >= 100) {
        tempoAnterior = tempoAtual;
        lerSensores();
        calcularPID();
    }
}
```

- 'delay()' congela todo o programa. O robô não responde a sensores durante esse tempo!
- Para tarefas periódicas, use 'millis()' e verificação de tempo.

Modularização: Além de Funções

- À medida que o código cresce (centenas ou milhares de linhas), organizar tudo em um único arquivo '.ino' fica impossível.
- A solução é **separar o código em múltiplos arquivos**, cada um com uma responsabilidade.
- Exemplo de um seguidor de linha modularizado:
 - 'config.h' → constantes, pinos, parâmetros globais.
 - 'motor.h' / 'motor.cpp' → tudo sobre controle de motores.
 - 'sensores.h' / 'sensores.cpp' → leitura e processamento de sensores.
 - 'pid.h' / 'pid.cpp' → cálculo do algoritmo PID.
 - 'bluetooth.h' / 'bluetooth.cpp' → comunicação sem fio.
- Cada módulo é **independente** e pode ser testado isoladamente.

Arquivos '.h' (Header) - Declarações

```
// motor.h
#ifndef MOTOR_H
#define MOTOR_H

#include <Arduino.h>

void inicializarMotor();
void pararMotores();
void controlarMotor(int velEsq, int velDir);

#endif
```

- Arquivo '.h' contém **declarações** e **protótipos** de funções.
- 'ifndef' / 'define' / 'endif' previnem inclusão duplicada (include guards).
- É o contrato: aqui está o que o módulo oferece.
- Quem quer usar motor, inclui 'motor.h' e pronto.

Arquivos '.cpp' - Implementação

```
// motor.cpp
#include "motor.h"
#include "config.h"

void inicializarMotor() {
    for (int i = 0; i < 4; i++) {
        pinMode(MOTOR_PINS[i], OUTPUT);
    }
    pararMotores();
}

void pararMotores() {
    analogWrite(PWM_LEFT, 0);
    analogWrite(PWM_RIGHT, 0);
}

void controlarMotor(int velEsq, int velDir) {
    analogWrite(PWM_LEFT, velEsq);
    analogWrite(PWM_RIGHT, velDir);
}
```

Organização em Pastas

Estrutura típica de um projeto modularizado:

```
meu_seguidor/  
    meu_seguidor.ino (setup() e loop())  
    src/  
        config/  
            config.h (constantes e pinos)  
        motor/  
            motor.h  
            motor.cpp  
        sensores/  
            sensores.h  
            sensores.cpp  
        pid/  
            pid.h  
            pid.cpp  
        bluetooth/  
            bluetooth.h  
            bluetooth.cpp
```

- Cada módulo em sua própria pasta.
- 'config.h' centraliza **todas** as constantes (pinos, velocidades, parâmetros).

O arquivo 'config.h'

```
// config.h - Centraliza TODAS as constantes
#ifndef CONFIG_H
#define CONFIG_H

// Pinos
const int LED_PIN = 22;
const int PWM_LEFT = 21;
const int PWM_RIGHT = 2;
const int MOTOR_PINS[] = {18, 19, 5, 4};

// Par metros PID
float KP = 0.5;
float KD = 0.4;

// Velocidade
int VEL_MAX = 80;

// Estados globais
bool start = false;
bool linhaPerdida = false;
```

Incluindo Módulos no 'ino'

```
// meu_seguidor.ino
#include "src/config/config.h"
#include "src/motor/motor.h"
#include "src/sensores/sensores.h"
#include "src/pid/pid.h"
#include "src/bluetooth/bluetooth.h"

void setup() {
    Serial.begin(115200);
    inicializarMotor();
    inicializarSensores();
    inicializarBluetooth();
}

void loop() {
    processarComandosBT();

    if (millis() - lastLoopTime < loopInterval) return;
    lastLoopTime = millis();

    lerSensoresQTR();
```

Benefícios da Modularização

- **Legibilidade:** código organizado e fácil de navegar.
- **Manutenibilidade:** problema em motor? Vai direto em 'motor.cpp'.
- **Reutilização:** use 'motor.h' e 'motor.cpp' em outro projeto.
- **Testes:** teste cada módulo isoladamente antes de integrar.
- **Trabalho em equipe:** um desenvolve 'motor.cpp', outro 'sensores.cpp', sem conflitos.
- **Escalabilidade:** adicione novos módulos (LED, buzzer, LoRa) sem mexer no código existente.
- **Performance:** o compilador só inclui o código que você realmente usa.

Anti-padrão: Sem Modularização

```
//      RUIM: tudo em um arquivo de 1000+ linhas
void setup() {
    // ... 50 linhas de inicializa o ...
}

void loop() {
    // ... 200 linhas de código misturado ...
    // sensores, câmeras, motor, BT... tudo aqui!
}

// ... 50 funções espalhadas sem organização ...
```

Consequências:

- Difícil encontrar uma função específica.
- Um bug pode estar em qualquer lugar.
- Impossível reutilizar código em outro projeto.
- Trabalhar em equipe é muito difícil.

Resumo: Estrutura de um Bom Programa

- ❶ **Defina estruturas** para agrupar dados relacionados (Motor, Sensor).
- ❷ **Quebre em funções:** cada função faz uma coisa bem.
- ❸ **Use mini IPOs:** cada função tem entrada → processamento → saída.
- ❹ **Mantenha o loop() limpo:** apenas chamadas de função.
- ❺ **Teste isoladamente:** valide cada função antes de integrar.
- ❻ **Evite delay():** use 'millis()' para tarefas periódicas.
- ❼ **Modularize com arquivos:** separe em '.h' / '.cpp' e organize em pastas.
 - Um programa estruturado assim fica fácil de entender, manter e depurar.
 - Você consegue trabalhar em equipe sem conflitos.
 - Adicionar novos recursos é trivial.
 - Seu código fica profissional e reutilizável.